

STUDENT ACADEMIC PERFORMANCE ANALYZER

Project Report

An offline academic analytics and reporting application for student-level and class-level performance monitoring

Version 1.1.0 • Electron Desktop Edition • MIT License

1. Executive Summary

The Student Academic Performance Analyzer is an offline-first academic analytics application designed to help educators and institutions enter, analyze, filter, and report student performance data. It combines subject-wise assessment marks, attendance, previous-semester performance, configurable academic settings, class analytics, personalized recommendations, and professional PDF reporting in one interface.

The current implementation is delivered as a standalone Windows desktop application through Electron. The core interface remains client-side HTML, CSS, and JavaScript, while Electron provides a native application window and local execution without an HTTP server. Student records and application settings are stored locally in browser-compatible storage, making the application suitable for offline demonstrations, classroom administration, and local academic analysis.

- Student-level academic analysis with configurable assessment components.
- Subject-wise performance calculation and arrear detection.
- Overall performance classification and attendance classification.
- CGPA-style grade-point calculation on a 10-point scale.
- Class Analytics with search and Department, Section, Batch, and Semester filters.
- PDF preview and local Save-as-PDF reporting.
- Configurable departments, batches, assessment maximums, mark limits, and UI themes.
- Large deterministic demo dataset covering 14 departments, 3 sections, and 2,940 students.
- Windows installer packaging with Desktop and Start Menu shortcuts.

2. Problem Statement

Academic performance data is often distributed across spreadsheets, paper records, or separate departmental systems. This makes it difficult to obtain a consistent view of a student's subject performance, attendance, semester context, and overall academic status.

- Manual calculation of subject percentages and overall performance can be repetitive and error-prone.
- Subject-level arrears may be missed when only overall averages are considered.
- Finding a particular student across departments, batches, sections, and semesters can be time-consuming.
- Class-level filtering and comparison are difficult when records are not organized consistently.
- Generating a professional report for a department, class, semester, or individual student usually requires additional manual formatting.
- Institutions may use different assessment maximums, departments, batches, and visual preferences.

3. Objectives

- Provide a simple interface for entering and updating student academic records.
- Calculate subject performance consistently from multiple assessment components.
- Combine academic performance with attendance to produce an overall performance indicator.
- Identify subject-wise arrears even when a student's overall score is above the arrear threshold.
- Support multiple semester records for the same Student ID.
- Provide searchable class analytics using academic filters.
- Generate previewable, structured PDF performance reports.
- Allow configuration without modifying source code.
- Operate completely offline as a Windows desktop application.

4. Project Scope

Area	Included in the application
Student management	Student ID, name, department, section, batch, semester, attendance and previous-semester percentage.
Assessment	Internal, Assignment, Unit Test and Final Semester Exam marks.
Analytics	Student-level analysis, class analytics, attendance status, performance status, CGPA and trend.
Arrears	Subject-level detection and visible arrear count/message.
Configuration	Departments, batches, assessment maximums, mark limits and UI themes.
Reporting	All-student, filtered-group and selected-student PDF reporting with preview.
Desktop	Electron-based offline Windows application and NSIS installer.
Data storage	Local browser-compatible storage; no remote database or backend server.

5. Target Users and Use Cases

User	Typical use
Faculty / Class Mentor	Enter marks and attendance, inspect student performance, identify weak subjects.
Department Staff	Filter students by department, section, batch and semester and produce reports.
Academic Coordinator	Review class-level averages, attendance and performance distributions.
Project Demonstrator	Demonstrate academic analytics, configurable settings and offline desktop packaging.
Student Support / Advisor	Review individual semester performance and generated reports.

6. System Architecture

The application follows a lightweight client-side architecture. The presentation layer is built with HTML and CSS; JavaScript provides application logic, calculations, filtering, storage, demo-data generation and report rendering. Electron wraps these local web assets in a desktop window. The application does not depend on a REST API, cloud database, or local HTTP server.

Layer	Technology / approach	Responsibility
Presentation	HTML5 + CSS3	Forms, dashboard, settings, analytics table, print modal and responsive UI.
Application logic	Vanilla JavaScript	Validation, calculations, filtering, CRUD-style updates, recommendations and reporting.
Local persistence	Web Storage / localStorage	Student records, settings, themes, batches and departments.
Desktop shell	Electron	Native Windows window, local file loading and desktop packaging.
Installer	electron-builder + NSIS	Windows installer, installation directory selection and shortcuts.
Reporting	Browser print / print-preview HTML	A4-style report layout and Save-as-PDF workflow.

7. Technology Stack

Technology	Role
HTML5	Application structure and semantic UI.
CSS3	Responsive layout, themes, cards, tables, dashboard and report styling.
JavaScript (ES6+)	Core application logic and calculations.
Electron 38.x	Standalone desktop application shell.
electron-builder 26.x	Windows application packaging.

NSIS	Installer target used for Windows setup.
Web Storage API	Offline local data persistence.
JSON	Demo dataset and export-oriented data representation.
GitHub / Git	Source-code hosting and version control target.

8. Data Model

A student record contains the following major fields:

Field	Description
Student ID	Unique identifier used to locate a student record.
Name	Student's display name.
Department	Academic department; built-in list can be extended.
Section	Class/section identifier such as A, B or C.
Batch	Academic batch such as 2025-29.
Semester	Semester number, currently represented as 1–8 in the interface.
Attendance	Attendance percentage from 0 to 100.
Previous Semester %	Previous-semester performance used for trend comparison.
Subjects	List of subject entries and four assessment marks per subject.

Semester is part of the record identity for practical duplicate handling. Therefore the same Student ID can legitimately occur in multiple semester records, while an accidental duplicate of the same student and same semester can be treated as an update/replacement case.

9. Assessment and Calculation Method

The default assessment maximums are configurable and currently start at:

Component	Default maximum
Internal	60
Assignment	10
Unit Test	15
Final Semester Exam	100

For each subject, each component is normalized against its configured maximum. The subject percentage is the average of the four normalized component percentages:

$$\text{Subject \%} = ((\text{Internal}/\text{Max}) + (\text{Assignment}/\text{Max}) + (\text{Unit Test}/\text{Max}) + (\text{Final Exam}/\text{Max})) / 4 \times 100$$

The application then combines the average subject performance with attendance. In the current implementation, subject performance contributes 85% and attendance contributes 15%:

$$\text{Overall \%} = \text{Subject Average} \times 0.85 + \text{Attendance} \times 0.15$$

Performance classification:

Overall percentage	Classification
70% and above	Good Performance
50% to below 70%	Less Performance
Below 50%	Arrear / Failed

Attendance classification:

Attendance	Status
90% and above	Good
75% to below 90%	Medium
Below 75%	Low

Subject-level arrear detection is independent of the overall performance label: every subject below 50% is counted as an arrear subject. This ensures a student can be classified as Good Performance overall while still being clearly identified as having one or more subject arrears.

Grade-point mapping used for the CGPA-style indicator:

Percentage	Grade point
90–100	10
80–89.99	9
70–79.99	8
60–69.99	7
50–59.99	6
40–49.99	5
Below 40	0

CGPA is calculated as the average of the subject grade points for the selected student record.

10. Personalized Recommendations

The application generates concise rule-based recommendations rather than using a machine-learning model. Rules inspect attendance, weak subject percentages, final-exam performance, internal performance, and assignment performance.

- Attendance below 75% produces an attendance improvement recommendation.
- The two weakest subjects below 60% are highlighted for additional practice.
- Low final-exam component performance triggers exam-preparation guidance.
- Low internal performance triggers topic review and teacher-feedback guidance.
- Low assignment performance triggers consistency and deadline guidance.
- If no issue is detected, the application recommends maintaining the current study routine.

11. Functional Modules

Module	Purpose
Student Entry & Update	Captures student identity, academic context, attendance, previous-semester performance and subject assessment marks.
Subject Assessment	Supports multiple subject cards with configurable maximum marks and automatic subject-percentage calculation.
Analysis Result	Displays overall score, CGPA, academic result, attendance, subject chart and recommendations.
Class Analytics	Provides searchable and filterable student listing with department, section, batch and semester controls.
Configuration	Controls departments, batches, assessment maximums, mark limits and themes.
PDF Reporting	Supports all-student, selected-group and selected-student reports, including semester-aware student selection.
Demo Data	Loads deterministic sample records across all configured departments.
Desktop Packaging	Provides an Electron window and NSIS installer for Windows.

12. Configuration Options

- Department Management — add or remove departments while retaining the protected default department set.
- UI Theme Customization — choose from multiple light, dark and accent-colour themes.
- Batch Management — add, edit/select and remove configurable academic batches where permitted.
- Assessment Configuration — change the default maximum marks for each assessment component.
- Custom Mark Limits — configure Internal, Assignment, Unit Test and Final Semester Exam limits used by validation and calculations.

13. Class Analytics

Class Analytics is the main class-level monitoring interface. It displays the number of matching students, class average, number of Good Performance records and average attendance, followed by a detailed student table.

- Search by Student ID / roll number, name, performance status and arrear-related terms.
- Filter by department.
- Filter by section.
- Filter by batch.
- Filter by semester.
- View individual student records.
- Clear individual records without needing to clear the complete dataset.
- Display multiple semester records for the same Student ID as separate rows.

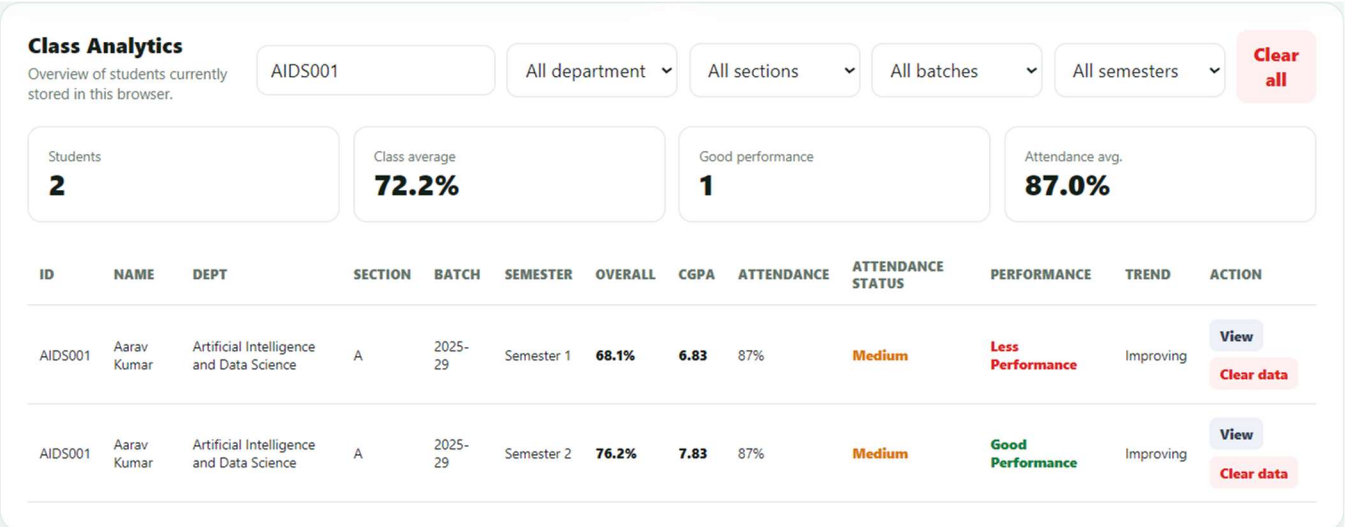


Figure 1. Class Analytics with department, section, batch and semester filtering.

PDF REPORT

×

Print Performance Report

Choose what you want to print or save as PDF.

Report scope

Selected student

▼

Search student (Name / Student ID)

Semester

aids001

All semesters

▼

Student

Aarav Kumar — AIDS001 — Artificial Intelligence and Data Science / A • Semester 1

▼

Aarav Kumar — AIDS001 — Artificial Intelligence and Data Science / A • Semester 1

Aarav Kumar — AIDS001 — Artificial Intelligence and Data Science / A • Semester 2

Cancel

Open Print / Save PDF

Figure 2. Selected-student PDF reporting with student search and semester selection.

14. PDF Reporting and Print Workflow

The reporting module is designed to turn stored academic records into a structured, print-ready report. The user first selects the report scope, then previews the generated report before saving it as a PDF.

Scope	Available selection
All students	All stored student records.
Selected department / group	Department, section, batch and semester filters, including All departments.
Selected student	Search by name or Student ID and choose All semesters or a specific semester.

- Student reports include identity, academic context, overall percentage, CGPA, result, attendance, subject average and subject performance.
- Subject arrears are shown explicitly.
- Arrear records receive a red visual indicator/border in the report.
- Reports use compact cards and grouped department/batch/semester/section structure.
- PDF preview is available before the final Save PDF action.

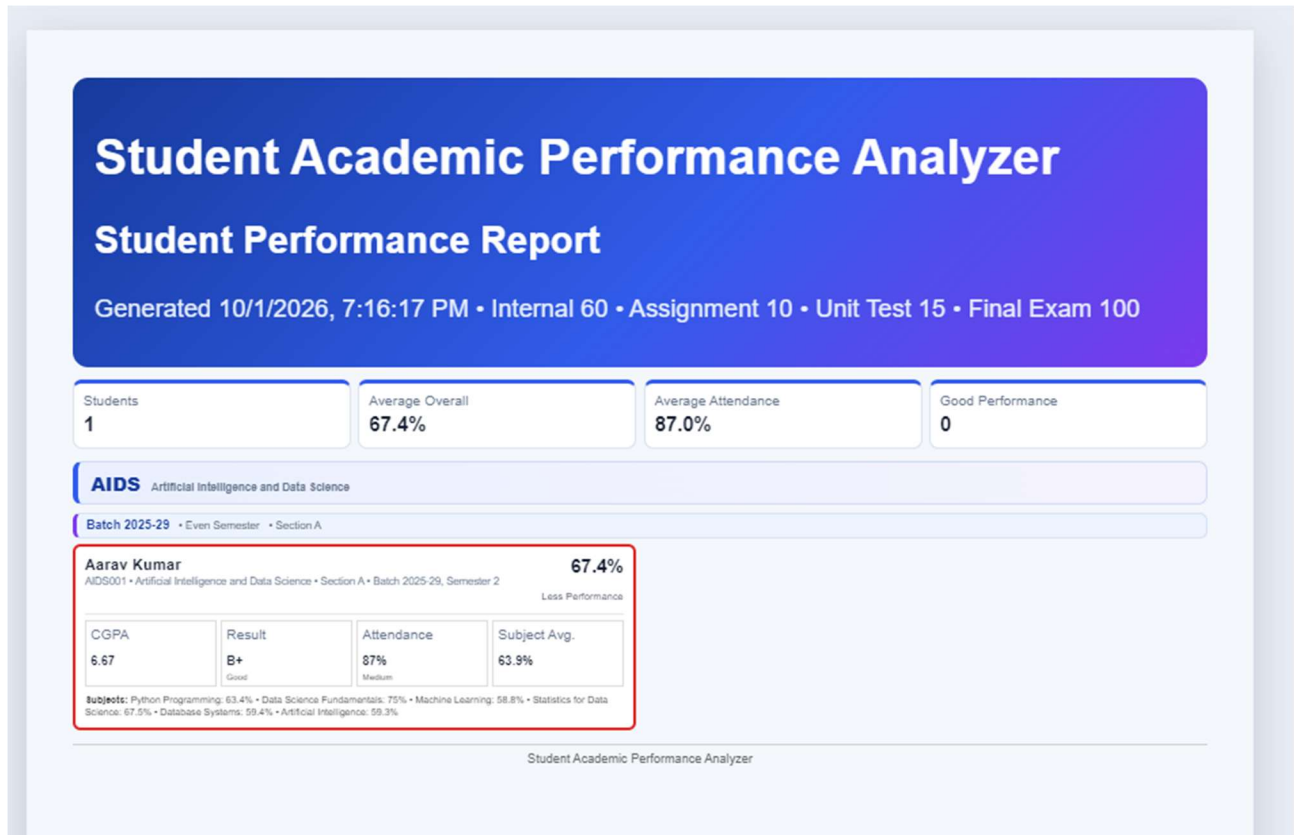


Figure 3. Generated student performance report preview.

15. Windows Desktop Deployment

The desktop edition uses Electron to load the application directly from local files. The project is packaged with electron-builder using the NSIS Windows installer target.

Configuration	Current value
Application ID	com.sapa.studentperformance
Product name	Student Academic Performance Analyzer
Electron entry	electron/main.js
Installer	NSIS
Architecture	Windows x64
Desktop shortcut	Enabled
Start Menu shortcut	Enabled
Installation directory selection	Enabled
Post-install launch	Enabled
Portable target	Not part of the current package configuration

Build workflow:

1. Install Node.js LTS on Windows.
2. Run “build-windows.bat” (it runs “npm install” and “npm run dist” commands to install dependencies and creates dist folder)
- (or)
3. Open the project folder in Command Prompt or PowerShell.
4. Run npm install and Run npm run dist.
5. Open the generated dist folder and run the NSIS setup executable.
6. Complete installation and launch the application from the Desktop or Start Menu.

16. Data Storage, Privacy and Offline Operation

Student records and configuration are stored locally using the Web Storage API. The application does not require a remote server, external database, or network connection for its core operation.

- Student data remains on the local machine/browser storage used by the application.
- Settings such as assessment maximums, themes, departments and batches are also stored locally.
- The desktop wrapper loads local application assets rather than calling a hosted backend.
- Users should still maintain their own backups or JSON exports when academic records are important, because local storage is not a replacement for institutional backup infrastructure.

17. Demo Dataset

The project includes a deterministic demo-data generator intended for testing and demonstrations. The current dataset design covers 14 departments, 3 sections per department, 70 students per section, and therefore 210 records per department.

Metric	Value
Departments	14
Sections per department	3 (A, B, C)
Students per section	70
Students per department	210
Total demo records	2,940
Default demo batch	2025-29
Semester range	1–8

Department-specific subject sets are used in the demo records, with deterministic variation in attendance, previous-semester performance and assessment marks so that analytics and reporting views contain realistic variation.

18. Current Subject Catalogue (Demos)

Code	Subject
U25ADG01	Digital Principles and Computer Organization
U25CSG11	Object Oriented Programming Using Java
U25CSG12	Data Structures
U25CSG13	Object Oriented Programming Using Java Laboratory
U25CSG16	Database Management Systems
U25CSG18	Database Management Systems Laboratory
U25ITG01	Software Engineering
U25MA301	Probability and Statistical Modeling for Computing Systems
U25MCC08	Professional Skill Development - I
U25MNC02	Essence of Indian Traditional Knowledge
U25OAM01	AWS Fundamentals
U25OCS04	Innovation Through Engineering Thinking

19. Testing and Validation Checklist

Test area	Expected result
Student creation	Enter a new student with valid identity, attendance and subject marks; verify analysis appears.
Assessment validation	Enter marks above configured maximums; verify the input limits prevent invalid values.
Dynamic settings	Change assessment maximums; verify labels and calculations update.
Semester records	Create the same Student ID in different semesters; verify both appear as separate records.
Duplicate semester	Submit the same student identity for the same semester; verify replacement/update handling.
Analytics filters	Filter by department, section, batch and semester; verify only

	matching records remain.
Analytics search	Search by Student ID, name and performance/arrear terms.
Arrear detection	Place a subject below 50%; verify arrear count/message and report styling.
Performance classification	Verify 70%+ Good, 50–69.99% Less Performance and below 50% Arrear/Failed.
Attendance classification	Verify <75 Low, 75–89.99 Medium and 90+ Good.
PDF student scope	Search a student and generate All semesters and specific-semester reports.
PDF group scope	Generate filtered reports using department, section, batch and semester.
PDF preview	Confirm the report can be reviewed before saving.
Theme persistence	Change theme and verify it persists locally.
Desktop build	Run npm run dist and verify the NSIS installer is produced.
Installer shortcuts	Install the application and verify Desktop and Start Menu shortcuts.

20. Current Limitations

- Local storage is intended for local/offline use and is not a multi-user institutional database.
- Authentication and role-based access control are not included in the current version.
- Cloud synchronization and centralized backup are not included.
- CGPA is a simplified subject-average grade-point indicator and may need adaptation to an institution's official credit-weighted CGPA rules.
- PDF output relies on the local print/save workflow rather than a dedicated server-side PDF engine.
- Browser mode and desktop mode share the client-side application logic; deployment should use the Windows installer for normal desktop use.

21. Future Enhancements

- Credit-weighted CGPA and institution-specific grading schemes.
- CSV/Excel import and export with validation.
- Encrypted local database or optional institutional database integration.
- User authentication and role-based access.
- Semester-to-semester longitudinal dashboards.
- Advanced charts for department and batch comparison.
- Automated attendance and mark import from institutional systems.
- Configurable report templates and institutional branding.
- Backup/restore management and scheduled local backups.
- Optional cloud synchronization for authorized institutional deployments.

22. Conclusion

The Student Academic Performance Analyzer provides a focused workflow for converting raw academic assessment and attendance data into understandable performance information. Its combination of configurable assessment settings, subject-level arrear detection, semester-aware student records, class analytics, personalized rule-based recommendations and structured PDF reporting addresses both individual student review and class-level monitoring.

The Electron desktop edition further packages the application as a standalone offline Windows tool, reducing deployment complexity for users who do not need a backend or internet connection. The architecture remains intentionally lightweight, making the project suitable for academic demonstration, local administration and future extension toward more advanced institutional data management.

Appendix A — Repository Structure

Path	Purpose
index.html	Main application interface.
css/style.css	Application and report styling.
js/app.js	Application logic, calculations, analytics, storage and reporting.
data/demo-students.json	Demo student data resource.
electron/main.js	Electron main-process window creation.
electron/preload.js	Electron preload bridge.
package.json	Node/Electron project configuration and Windows packaging.
build-windows.bat	Windows build helper.
README.md	Project overview and usage documentation.
README-DESKTOP.md	Desktop build documentation.

Appendix B — Quick Start

Development: npm install → npm start

Windows installer: Run “build-windows.bat” (npm install → npm run dist → open the generated NSIS setup EXE in dist/)

License: MIT, with third-party dependency licenses remaining applicable where relevant.